ScrumMate

Project Team

Mishal Ali 22I-1291
Ayaan Mughal 22I-0861
Muaz Ahmed 22I-0832

Session 2022-2026

Supervised by

Ms. Marium Hida

Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

June, 2022

Contents

List of Figures

List of Tables

Acknowledgments

We would like to express our deepest gratitude to our supervisor, Ms. Mariam Hida, for her unwavering guidance, valuable insights, and continuous encouragement throughout the development of ScrumMate. Her expertise in Agile methodologies and AI systems greatly shaped the direction of this project.

We are also thankful to the Final Year Project Committee and the evaluation panel for their constructive feedback and support during the reviews and presentations. Their suggestions helped refine the system and improve the quality of this report.

Our sincere appreciation goes to the Department of Computer Science at the National University of Computer and Emerging Sciences, Islamabad, for providing the necessary resources and an excellent research environment.

Finally, we are forever grateful to our families and friends for their patience, motivation, and unconditional support during the many hours dedicated to this project.

Abstract

Modern Agile software development teams face inefficiencies due to manual coordination, inconsistent documentation, and communication gaps in Scrum ceremonies. This project presents ScrumMate, an AI-driven multi-agent system that automates sprint planning, daily standups, backlog management, task assignment, and reporting. The system uses Whisper for real-time transcription with speaker diarization, fine-tuned LLMs (Ollama) for extracting meeting minutes, action items, and user stories, and a hybrid skill-based algorithm for equitable task allocation. Additional modules generate test cases, detect blockers, manage change requests, and provide semantic search via a vector database. The system was implemented following an event-driven microservices architecture with LangGraph for agent orchestration and n8n for workflow automation. Testing comprised 35 unit tests achieving 94.3% pass rate and 85% code coverage, confirming functional reliability and performance within defined thresholds. External integrations with Jira, Trello, and GitHub were partially realized, with OAuth synchronization requiring further refinement. ScrumMate successfully transforms meeting conversations into structured, actionable project artifacts, reducing manual overhead and enabling Scrum Masters to focus on strategic decisions. Future work includes completing OAuth2.0 integrations, optimizing the assignment algorithm for larger teams, and deploying on Kubernetes with health monitoring.

Chapter 1

Introduction

ScrumMate is an AI-driven Scrum Master multi-agent system designed to optimize and automate key processes in Agile software development (1; 2). It functions as an intelligent assistant that facilitates sprint planning, backlog refinement, daily standups, retrospectives, and project reporting. By leveraging conversational AI, real-time transcription, and autonomous task coordination, ScrumMate minimizes manual administrative work and ensures that development teams stay synchronized, productive, and data-informed. It integrates seamlessly with tools such as Git repositories, issue trackers, CI/CD systems, and databases to maintain transparency, traceability, and continuous feedback throughout the software lifecycle. Through natural language interaction and an adaptive dashboard, ScrumMate provides real-time insights, identifies bottlenecks, and supports informed decision-making.

The primary readers and users of ScrumMate include several groups involved in the software development process. **Developers** benefit from the system's automated task tracking, sprint summaries, and intelligent reminders. **Project managers and product owners** use ScrumMate to monitor team performance, manage backlogs, and maintain alignment with sprint goals. **Testers and QA engineers** rely on its progress reports and meeting summaries to validate deliverables and ensure quality consistency. **End users and Scrum teams** experience streamlined communication and reduced management overhead through AI-assisted coordination. **Documentation writers and trainers** can utilize automatically generated meeting notes and reports for creating accurate user guides and training material. Finally, **marketing and business stakeholders** gain a high-level overview of project progress and productivity metrics, supporting better planning and communication across teams.

1.1 Existing Solutions

Agile software development teams increasingly rely on AI-powered tools to automate Scrum ceremonies, reduce manual overhead, and improve collaboration. Several commercial and open-source solutions exist, ranging from general-purpose meeting assistants to specialized AI Scrum Masters. However, most address only isolated aspects of the Scrum workflow - transcription, task creation, or reporting - rather than providing an integrated, end-to-end solution. This section reviews relevant existing systems, evaluates their capabilities, and identifies their limitations to establish the rationale for ScrumMate.

Table 1.1: Comparison of Existing Solutions

System Name	System Overview	System Limitations
Spinach.io (3)	AI Scrum Master (GPT-4) joins meetings, generates instant summaries and action items, updates Jira. Atlassian-backed.	No skill-based task assignment; no test case generation; paid; limited AI agent customization.
Notion (AI + Integrations) (4)	AI Meeting Notes transcribes conversations, extracts decisions, drafts action items. Customizable AI agents (Notion 3.0).	No native audio ingestion; manual task-board sync; no test generation; AI Agents lack scheduled triggers ("coming soon").
AIPM (5)	Vertical AI Scrum Master for 20-200 person teams; runs daily standups, updates Jira, always-on. 10x cheaper than human Scrum Master.	No built-in meeting transcription (relies on external tools); no test generation; minimal reporting beyond standups; no backlog decomposition or skill-based assignment.
SprintBot (6)	AI Scrum copilot joins Zoom standups, detects anti-patterns (hedge words, blockers, meeting drift) using Gemini 2.5 Flash; auto-creates Jira tickets; meeting health scoring (0-100).	Standup-only (no sprint planning, retrospectives, backlog); no skill-based assignment; no test generation; limited to Zoom.

Continued on next page

System Name	System Overview	System Limitations
Tusk (7)	Generates and runs unit/integration tests autonomously; self-healing test cases from codebase and business context.	No Scrum or meeting functionality; no task assignment or backlog management; pure testing tool, not an agile assistant.
Zapier AI (8)	Workflow automation with AI steps, agents, chatbots; connects 1000s of apps (Jira, Trello, Slack); conditional logic.	No native meeting transcription or summarization; no built-in Scrum intelligence; manual workflow design; no skill-based task assignment.
Otter.ai (9)	AI meeting assistant with real-time transcription, speaker identification, enterprise search across Jira/Notion/Salesforce via MCP.	No Scrum-specific intelligence (standups, retrospectives, backlog); no action item assignment; no test generation; Jira integration limited to search, not auto task creation.

Among the systems reviewed, Spinach.io is the most comprehensive AI Scrum assistant, providing meeting transcription, action item extraction, and Jira synchronization. However, it lacks built-in skill-based task assignment and test case generation - capabilities that ScrumMate addresses through its modular, multi-agent design. SprintBot makes notable advances in anti-pattern detection and meeting health scoring, but its focus is narrowly limited to daily standups. General-purpose tools such as Zapier and Notion require extensive manual configuration to function as dedicated Scrum assistants.

In contrast, ScrumMate distinguishes itself through a multi-agent architecture that integrates real-time transcription (10), natural language understanding for action item and user story extraction (11), task decomposition and backlog management, skill-based optimal task assignment, automated standup processing, test case generation, and semantic knowledge retrieval (12; 13). No existing solution combines all these capabilities within a unified, event-driven framework designed for end-to-end Scrum automation.

1.2 Problem Statement

Modern software development teams face increasing pressure to deliver faster, maintain higher quality, and stay aligned across distributed environments. While the Agile and Scrum frameworks provide structure for iterative development, their effectiveness heavily depends on human coordination, consistency, and timely communication.

In many teams, Scrum Masters and project managers spend a significant portion of their time on repetitive administrative duties - organizing meetings, updating boards, writing reports, and ensuring adherence to Scrum practices - rather than focusing on strategic improvement. Communication gaps, manual documentation, missed updates, and inconsistent sprint reviews often lead to inefficiencies, delayed deliveries, and decreased team productivity. Remote and hybrid work environments further complicate coordination, making it difficult to track verbal commitments and progress updates shared in meetings.

This project develops ScrumMate, an AI-driven solution that automates and assists in Scrum-related tasks. The system reduces human dependency for routine management and ensures that essential Scrum events are captured, analyzed, and acted upon intelligently. It leverages real-time transcription and language models to understand team discussions, extract tasks and blockers, and automatically update project artifacts.

By minimizing errors from manual note-taking, ScrumMate ensures that every team member remains informed and accountable. It integrates with existing development tools, bridging the gap between meetings and actionable project updates while maintaining transparency and consistency. Ultimately, the system enhances team performance, streamlines sprint cycles, and allows Scrum Masters to focus on team growth, decision-making, and value delivery rather than repetitive coordination. In doing so, ScrumMate transforms the Scrum process into a smarter, data-driven workflow that supports productivity and continuous improvement.

1.3 Scope

ScrumMate encompasses the design and development of an AI-driven Scrum Master multi-agent system that automates, assists, and optimizes Agile project management workflows. The system facilitates all key Scrum ceremonies - sprint planning, backlog refinement, daily standups, sprint reviews, and retrospectives - by using natural language processing and intelligent automation to capture discussions and translate them into actionable data.

The core functionality of ScrumMate includes real-time meeting transcription (10), task extraction, automated documentation generation, and progress tracking through an integrated dashboard. The system analyzes team conversations to identify action items, update backlogs, and provide project status summaries without manual input. It also supports intelligent recommendations for workload balancing, priority adjustments, and sprint forecasting.

Through integration with GitHub, Jira, and CI/CD systems, ScrumMate maintains continuous synchronization between discussions, code changes, and task boards. The modular architecture enables role-based access for developers, project managers, testers, and

stakeholders. Key technical components - Whisper for speech-to-text (10), Ollama LLMs for reasoning (11), FAISS/Chroma for vector memory (12; 13), and n8n for workflow automation (14) - work together as a context-aware assistant.

The web interface built with React (15) and FastAPI (16) provides intuitive visualization and seamless collaboration. While the system does not replace human decision-making, it serves as a virtual Scrum assistant that ensures accountability and eliminates redundant manual effort.

The project boundaries focus on Scrum-based software development teams, emphasizing coordination and automation rather than full project management or code analysis. Future extensions may include additional methodologies or predictive analytics, but the current scope is limited to AI-assisted Scrum practices. Within these boundaries, ScrumMate aims to enhance team productivity, strengthen process adherence, and transform traditional Agile operations into an intelligent, automated workflow.

1.4 Modules

The system is decomposed into eight interconnected modules that collectively enable intelligent Scrum management, automation, and collaboration. Each module addresses a distinct aspect of the Scrum lifecycle and integrates with others through well-defined interfaces, ensuring seamless data flow and contextual understanding.

1.4.1 Module 1: Meeting Intelligence Agent

This module captures and transcribes team meetings, standups, and discussions in real time using speech-to-text technology (10). It serves as the entry point for data collection, converting spoken communication into structured digital records.

1. Converts live or recorded speech into accurate text using Whisper with speaker diarization.

2. Analyzes participation metrics (speaking time, engagement) and detects topic adherence.

3. Supports batch transcription of uploaded audio files and streams data to AI agents in real time.

1.4.2 Module 2: MoM, User Story & Action Item Generator

This module interprets transcribed content using large language models (11) to extract actionable insights, decisions, and summaries. It forms the cognitive layer of ScrumMate by simulating the analytical role of a Scrum Master.

1. Generates structured meeting minutes (MoM) and extracts user stories with acceptance criteria.

2. Identifies specific action items including owners, deadlines, and priorities.

3. Provides human-in-the-loop review for all generated documentation before finalization.

1.4.3 Module 3: Task Decomposition & Backlog Preparation

This module automates the breakdown of user stories into fine-grained tasks and manages the product backlog. It ensures that every extracted action item becomes a traceable work unit.

1. Decomposes user stories into categorized tasks with complexity, urgency, and dependency metadata.

2. Synchronizes backlog items bidirectionally with external trackers such as Jira and GitHub.

3. Maintains version history and audit trails for all backlog changes.

1.4.4 Module 4: Skill-Based Task Assignment

This module assigns tasks to developers based on skills, availability, and historical performance. It ensures equitable workload distribution while optimizing team velocity.

1. Maintains developer profiles tracking skills, velocity, and task history.

2. Implements a hybrid assignment algorithm combining rule-based logic and LLM reasoning.

3. Provides workload visualization and allows manual override by Scrum Masters.

1.4.5 Module 5: Stand-up Automation & Collaboration Sync

This module automates daily standup collection, analysis, and reporting (14). It handles both synchronous and asynchronous updates to keep teams aligned.

1. Collects standup updates ("yesterday, today, blockers") via voice or text input.

2. Generates concise standup summaries and automatically links developers with shared blockers.

3. Supports real-time collaboration sync showing team progress and dependencies.

1.4.6 Module 6: Testing, Evaluation & Sprint Review

This module automates test case generation, execution, and sprint analytics. It bridges the gap between development and quality assurance.

1. Automatically generates unit and integration test cases from task specifications.

2. Generates sprint analytics including burndown charts, velocity reports, and retrospective comparisons.

3. Integrates test results directly with task status updates.

1.4.7 Module 7: Blockers Identification & Removal

This module continuously monitors communication channels and project artifacts to identify and resolve blockers. It escalates critical issues and maintains historical data for pattern analysis.

1. Classifies blockers as internal or external and assigns appropriate resolution workflows.

2. Automatically notifies responsible parties and escalates critical blockers to Scrum Masters.

3. Tracks blocker resolution status and provides real-time updates to affected team members.

1.4.8 Module 8: Change Management System

This module handles change requests raised during meetings, evaluates their impact, and updates sprint plans accordingly. It ensures that scope changes are managed transparently.

1. Evaluates impact of change requests on scope, cost, and deadlines.
2. Generates impact analysis reports and suggests revised sprint plans.
3. Maintains change request audit trails with approval history and stakeholder notifications.

1.5 Work Division

The development of ScrumMate was carried out by a team of three members. The following table outlines the division of responsibilities across the eight modules and supporting tasks. All members jointly contributed to Module 4 (Hybrid Assignment Method Design and Testing) as well as final integration and deployment.

Table 1.2: Team Member Responsibilities

Team Member	**Student ID**	**Responsibility / Module / Feature**
Muaz Ahmed	22i-1125	Module 1: Audio pipeline (Noise Reduction, Whisper, Diarization) Module 2: LLaMA Fine-tuning for MoM/Stories/Action Items Module 3: n8n Integration with Jira/Trello Module 4: Joint development of assignment algorithm Module 5: Chatbot (collaboration with Mishal), Product Owner Interaction Module 6: Report Formation + Sprint Analytics Module 7: n8n Integration for Blocker Updates/Docs/Notifications Module 8: Revised Deadlines & Re-planning Workflow
Mishal Ali	22i-1291	Module 1: OpenCV Speaker Recognition, LangChain Lead, Metrics + Follow-up Questions Module 2: Minutes of Meeting Generation Module 2: Action Item & User Story Extraction (shared with Ayaan) Module 3: Task Generation from Stories/Action Items Module 4: Joint development of assignment algorithm Module 5: Chatbot (core dev, collaboration with Muaz), Developer-side Daily Updates Module 6: Test Case Generation Module 7: Chatbot for Blocker Detection from Conversations + Code (with Ayaan) Module 8: Impact/Cost Analysis
Ayaan Mughal	22i-0861	Module 1: Database Integration + Google Meet Integration Module 2: Minutes of Meeting Design + Action Items/User Stories Extraction (shared with Mishal) Module 3: Backlog Maintenance in Database Module 4: Joint development of assignment algorithm Module 5: DB Updates + n8n Integration, Historical Report File Creation Module 6: Chart Formation (Burn-downs, Visual Reports) Module 7: Database Updates for Blockers + Collaboration Logic (shared with team) Module 8: Risk Analysis
All Members		Module 4: Hybrid Assignment Method Design + Testing Final Integration & Deployment

1.5.1 User Classes and Characteristics

ScrumMate serves multiple user classes within an Agile software development environment. Each class has distinct responsibilities, interaction frequencies, and technical expertise requirements. Table 1.3 describes the primary user classes and their characteristics.

User Class	Description
Developers	Developers are the core users who interact with ScrumMate during sprint execution. They rely on the system for receiving task assignments, viewing sprint goals, and accessing summaries of daily standups. Developers have moderate to high technical expertise and use the platform daily.
Scrum Masters / Project Managers	Scrum Masters use ScrumMate to automate scheduling, track progress, generate reports, and monitor team productivity. They rely heavily on AI-generated insights and visual dashboards.
Product Owners	Product Owners use ScrumMate to review backlog changes, approve AI-suggested priorities, and analyze sprint summaries. Their interaction is critical during sprint planning and review phases.
Testers / QA Engineers	Testers utilize ScrumMate to access task updates, track bugs, and verify user stories. They depend on meeting summaries and automated test-related action extraction.
End Users / Scrum Team Members	All team members use the system to record discussions, receive summaries, and ensure clear communication across remote or hybrid teams.
Documentation Writers / Trainers	Documentation specialists use automated meeting notes and summaries to create user guides and training materials.
Business Stakeholders / Clients	Stakeholders access visual dashboards and AI-generated performance summaries for high-level project overviews.
System Administrators	Administrators manage user accounts, access permissions, data backups, and integrations. They require technical expertise in deployment and database management.

Table 1.3: User Classes and Characteristics for ScrumMate

Chapter 2

Project Requirements

This chapter defines the complete set of requirements that govern the behavior, performance, and constraints of ScrumMate. The requirements are organized into two main categories: functional requirements, which describe the system's core operations including meeting transcription (10), action item extraction, task management, and sprint automation; and non-functional requirements, which specify quality attributes such as reliability, usability, performance, security, and scalability. Together, these requirements ensure that ScrumMate functions as an intelligent, AI-driven Scrum assistant capable of automating key Agile processes while maintaining consistency, transparency, and adaptability across diverse development environments.

2.1 Use-cases

This section describes how users interact with ScrumMate through a set of well-defined use cases. A use case represents a specific interaction between an actor (such as a Developer, Scrum Master, Product Owner, or System Admin) and the system, detailing the steps involved and the expected system responses. Each use case includes a unique identifier, a description, the primary actor, preconditions, a main flow of events, alternate/exception flows, and postconditions.

For ScrumMate, twenty-one use cases have been identified, covering the entire Scrum lifecycle: meeting recording and transcription (UC01), action item extraction (UC02), backlog creation and updates (UC03), sprint planning and allocation (UC04), daily standup summarization (UC05), retrospective analysis (UC06), report generation (UC07), external integrations (UC08), test/QA synchronization (UC09), contextual search (UC10), conversational assistant (UC11), workflow automation (UC12), user and access management (UC13), system monitoring and logging (UC14), speaker training (UC15), backup and restore (UC16), scheduled meetings (UC17), test case generation (UC18), assignment

algorithm tuning (UC19), change management (UC20), and sprint re-planning (UC21). A use case diagram is provided in Figure 2.1 to visually summarize the relationships between actors and use cases, with actor generalization (Team Member, Product Owner, System Admin) to reduce complexity while maintaining complete functional coverage.

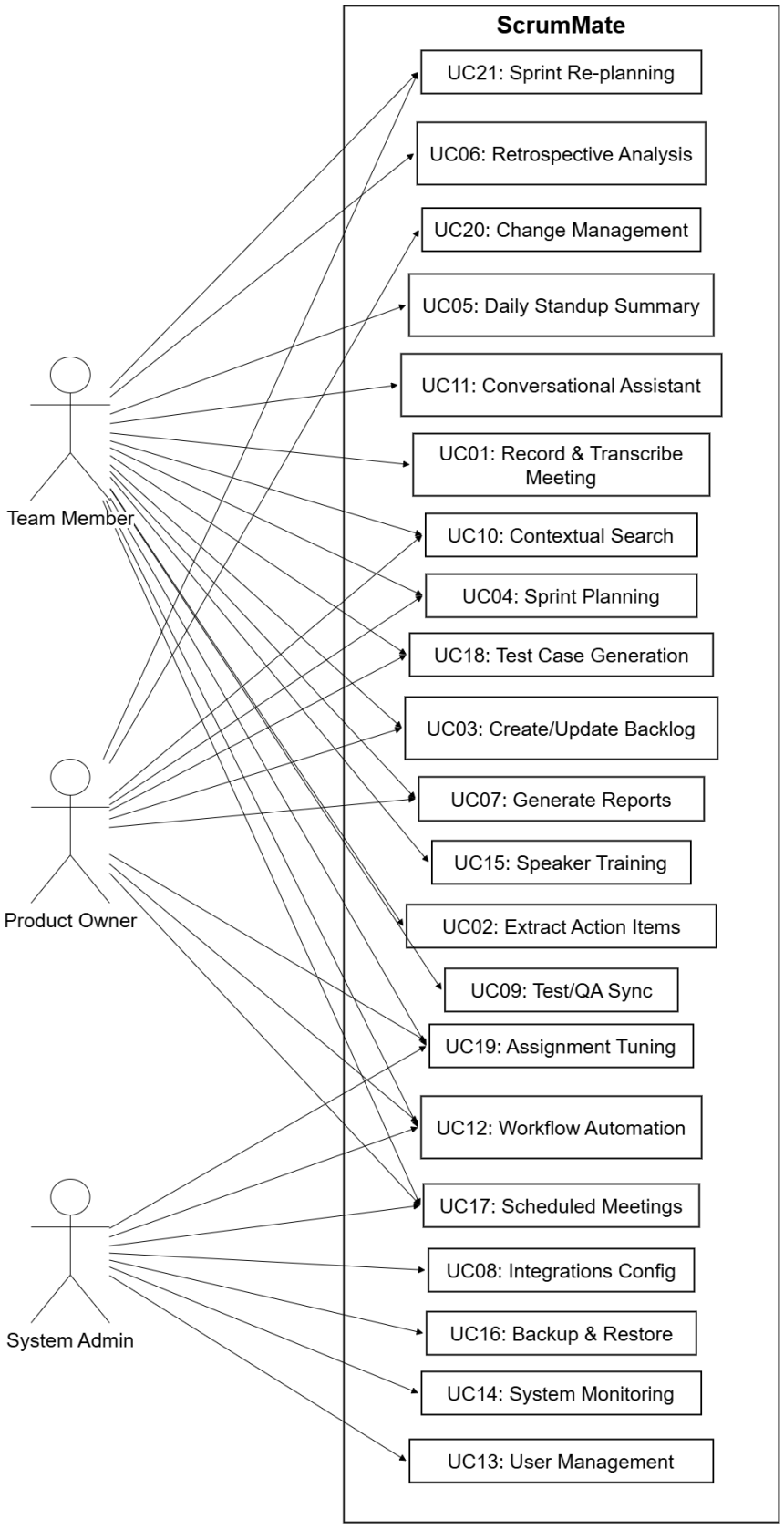

Figure 2.1: Use Case Diagram for ScrumMate

Note on Actor Generalization:
To simplify the use case representation and make the diagram more readable, multiple specific roles have been consolidated into three generalized actors:

- **Team Member** - represents the combined responsibilities of the Scrum Master, Developer, and QA Engineer.
- **Product Owner** - represents the combined roles of the Product Owner, Stakeholder, and Project Manager.
- **System Admin** - represents the combined responsibilities of the System Administrator and AI Agent.

This generalization maintains the same functional coverage while reducing visual complexity and ensuring that all interactions from the use cases remain represented.

2.2 Functional Requirements

This section describes the functional requirements of ScrumMate organized by the eight core modules defined in the system architecture. Each requirement is stated as a clear, testable obligation. The requirements align directly with the use cases described above.

2.2.1 Module 1: Meeting Intelligence Agent

Following are the requirements for Module 1:

1. FR1.1: The system shall allow authenticated users to start and stop meeting recording sessions from the project interface.
2. FR1.2: The system shall transcribe live audio to text in near real-time using speech-to-text engines (10), producing visible transcript streams.
3. FR1.3: The system shall perform speaker diarization and attach speaker labels with timestamps to transcript segments.
4. FR1.4: The system shall analyze participation metrics including speaking time and engagement levels.
5. FR1.5: The system shall detect topic adherence and identify off-topic conversation drift during meetings.

6. FR1.6: The system shall support batch transcription of uploaded audio files when live recording is unavailable.

7. FR1.7: The system shall persist transcripts with metadata to storage and index them for downstream processing.

2.2.2 Module 2: MoM, User Story & Action Item Generator

Following are the requirements for Module 2:

1. FR2.1: The system shall generate structured meeting minutes (MoM) from transcribed discussions.

2. FR2.2: The system shall extract user stories and epics from meeting dialogue.

3. FR2.3: The system shall identify specific action items with owners and deadlines from conversations.

4. FR2.4: The system shall provide human-in-the-loop review for generated documentation before finalization.

2.2.3 Module 3: Task Decomposition & Backlog Preparation

Following are the requirements for Module 3:

1. FR3.1: The system shall decompose user stories into fine-grained, categorized tasks.

2. FR3.2: The system shall annotate tasks with metadata including complexity, urgency, dependencies, and required skills.

3. FR3.3: The system shall organize tasks hierarchically into structured product backlogs.

4. FR3.4: The system shall synchronize backlog items with external trackers (Jira, GitHub, Trello) using configurable mappings.

5. FR3.5: The system shall support manual editing and reorganization of backlog items by authorized users.

6. FR3.6: The system shall maintain version history and audit trails for backlog changes.

2.2.4 Module 4: Skill-Based Task Assignment

Following are the requirements for Module 4:

1. FR4.1: The system shall maintain developer profiles tracking skills, velocity, and task history.

2. FR4.2: The system shall implement a hybrid assignment algorithm combining rule-based logic and LLM reasoning (11).

3. FR4.3: The system shall ensure equitable workload distribution while matching task requirements to developer skills.

4. FR4.4: The system shall propose task assignments during sprint planning with justification for recommendations.

5. FR4.5: The system shall allow manual override of automated assignments by Scrum Masters.

6. FR4.6: The system shall provide workload visualization showing current assignments and capacity per team member.

2.2.5 Module 5: Stand-up Automation & Collaboration Sync

Following are the requirements for Module 5:

1. FR5.1: The system shall collect daily stand-up updates (“yesterday, today, blockers”) via voice or text input.

2. FR5.2: The system shall automatically link developers with shared issues or blockers.

3. FR5.3: The system shall generate concise stand-up summaries highlighting progress and impediments.

4. FR5.4: The system shall provide real-time collaboration sync showing team progress and dependencies.

5. FR5.5: The system shall support asynchronous stand-ups for distributed teams.

6. FR5.6: The system shall persist stand-up data for performance tracking and historical analysis.

2.2.6 Module 6: Testing, Evaluation & Sprint Review

Following are the requirements for Module 6:

1. FR6.1: The system shall automatically generate test cases from task specifications and requirements.
2. FR6.2: The system shall execute unit and integration tests, logging results and coverage metrics.
3. FR6.3: The system shall generate sprint analytics including burn-down charts and velocity reports.
4. FR6.4: The system shall compile sprint outcome summaries with backlog recommendations for product owners.
5. FR6.5: The system shall support retrospective analysis by comparing current and historical sprint data.
6. FR6.6: The system shall integrate test results with task status updates automatically.

2.2.7 Module 7: Blockers Identification & Removal

Following are the requirements for Module 7:

1. FR7.1: The system shall continuously monitor communication channels and project artifacts for blockers.
2. FR7.2: The system shall classify blockers as internal or external and assign appropriate resolution workflows.
3. FR7.3: The system shall automatically notify responsible parties or schedule interventions for blocker resolution.
4. FR7.4: The system shall track blocker resolution status and provide real-time updates to affected team members.
5. FR7.5: The system shall escalate critical blockers to Scrum Masters and project managers based on severity.
6. FR7.6: The system shall maintain historical blocker data for pattern analysis and preventive measures.

2.2.8 Module 8: Change Management System

Following are the requirements for Module 8:

1. FR8.1: The system shall evaluate the impact of change requests on scope, cost, and deadlines.
2. FR8.2: The system shall generate impact analysis reports for proposed changes.
3. FR8.3: The system shall suggest revised sprint plans and deadlines when changes are approved.
4. FR8.4: The system shall support re-planning workflows that adjust backlogs and task assignments.
5. FR8.5: The system shall maintain change request audit trails with approval history.
6. FR8.6: The system shall notify all stakeholders of approved changes and their impacts.

2.2.9 Cross-Module Requirements

Following are requirements that span multiple modules:

1. FR9.1: The system shall provide role-based access control with authentication for all users.
2. FR9.2: The system shall maintain integration with communication platforms and issue trackers.
3. FR9.3: The system shall provide a web dashboard with real-time metrics and management interfaces.
4. FR9.4: The system shall implement backup and recovery procedures for all project data using PostgreSQL (17).
5. FR9.5: The system shall support containerized deployment (18) with health monitoring and logging.

2.3 Non-Functional Requirements

This section specifies the quality requirements for ScrumMate to ensure it meets performance, reliability, and usability standards in real-world Agile team environments. Each requirement is specific, quantitative, and verifiable.

2.3.1 Reliability

1. NFR-1: The system shall maintain 99.9% uptime during business hours for core meeting and task management functions.

2. NFR-2: No meeting transcripts or generated action items shall be lost due to system failures.

3. NFR-3: The system shall automatically recover from integration failures with external tools (Jira, Slack) within 5 minutes.

2.3.2 Usability

1. NFR-4: New users shall be able to start their first meeting recording and generate summaries within 5 minutes without training.

2. NFR-5: Common actions like approving AI suggestions or assigning tasks shall be completable in two clicks or less.

3. NFR-6: The dashboard shall provide real-time visual feedback for all user interactions.

4. NFR-7: Error messages shall be clear and provide actionable steps for resolution.

2.3.3 Performance

1. NFR-8: Meeting transcription shall display within 8 seconds after the meeting stops.

2. NFR-9: AI extraction of action items and user stories shall complete within 30 seconds after meeting ends.

3. NFR-10: Dashboard pages shall load within 3 seconds under normal network conditions (20 Mbps or faster).

4. NFR-11: Task assignment algorithms shall process and suggest allocations within 10 seconds for teams of up to 15 members.

5. NFR-12: Semantic search across meeting history using vector embeddings (12; 13) shall return results within 2 seconds.

2.3.4 Security

1. NFR-13: All user data and meeting transcripts shall be encrypted both in transit (TLS 1.2+) and at rest (AES-256).
2. NFR-14: Role-based access control shall prevent unauthorized access to project data and admin functions.
3. NFR-15: User authentication sessions shall expire after 24 hours of inactivity.
4. NFR-16: All system access and data modifications shall be logged with timestamps and user identities for audit purposes.

2.3.5 Scalability

1. NFR-17: The system shall support up to 50 concurrent meeting recording sessions without degradation.
2. NFR-18: Database query response time shall remain under 500ms for up to 100,000 stored transcripts.
3. NFR-19: The system shall support horizontal scaling of agent services through container orchestration (18).

2.3.6 Maintainability

1. NFR-20: The system shall provide structured logs for all agent activities and API calls.
2. NFR-21: New AI models shall be deployable without modifying core agent orchestration logic.
3. NFR-22: The codebase shall maintain test coverage above 80% for critical modules.

2.3.7 Compatibility

1. NFR-23: The system shall function on major browsers (Chrome, Firefox, Safari, Edge) released within the last two years.
2. NFR-24: Audio processing shall support standard formats (MP3, WAV, M4A) from common meeting platforms (Zoom, Google Meet, Microsoft Teams).
3. NFR-25: The system shall provide REST APIs compatible with external integration tools.

2.3.8 Data Management

1. NFR-26: Users shall be able to export their project data in standard formats (PDF, CSV, JSON).
2. NFR-27: Meeting recordings and transcripts shall be retained according to configurable project retention policies (default: 12 months).
3. NFR-28: The system shall support automated daily backups of all databases and object storage.

Chapter 3

System Overview

ScrumMate is an AI-driven Scrum management platform designed to assist software development teams in automating and streamlining the Scrum process through intelligent, context-aware agents (2). The system integrates real-time meeting transcription, natural language understanding, task extraction, sprint planning, reporting, and workflow automation into a unified platform. By combining speech-to-text technology (10), large language models (11), and semantic search capabilities (12; 13), ScrumMate functions as a virtual Scrum Master that listens, understands, and acts upon team discussions to generate actionable insights. It supports multiple user roles - Developers, Scrum Masters, Product Owners, Testers, and Administrators - providing each with personalized tools and dashboards for collaboration and decision-making. Built as a modular, web-based application, ScrumMate emphasizes scalability, privacy, and interoperability with existing development ecosystems such as Jira, GitHub, Slack, and CI/CD pipelines, thereby improving sprint visibility and ensuring continuous traceability from meetings to tasks.

3.1 Architectural Design

The architecture of ScrumMate follows an event-driven microservices pattern to support scalable, decoupled agent operations. This approach allows each functional module to operate independently while communicating asynchronously through a message queue. The system is decomposed into specialized AI agents (Scheduler, Collector, Facilitator) that collaborate to automate meeting processing, task extraction, and sprint management. This decomposition ensures loose coupling, independent scalability, and resilience against individual component failures.

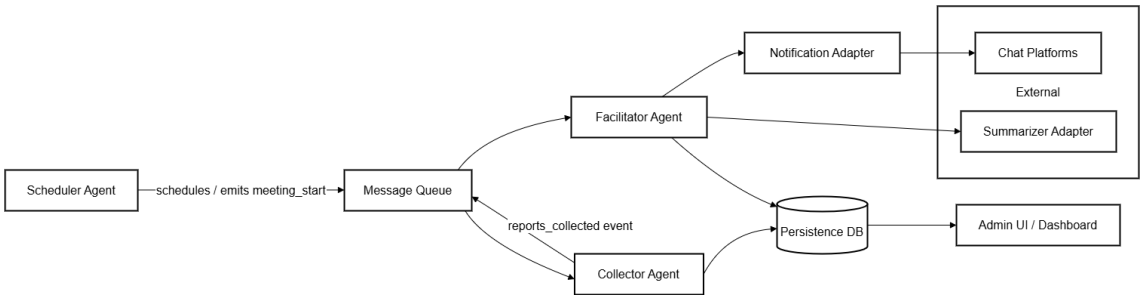

Figure 3.1: Box and Line Diagram: High-level component flow

The box and line diagram (Figure 3.1) illustrates the primary components and their interactions. The Scheduler Agent automatically triggers meetings and initializes the Scrum workflow via the Message Queue. The Collector Agent gathers standup reports, meeting audio, and team updates. The Facilitator Agent processes collected data, generates summaries, extracts action items, and updates the Persistence DB. All agents decouple through the Message Queue, enabling independent scaling and fault isolation.

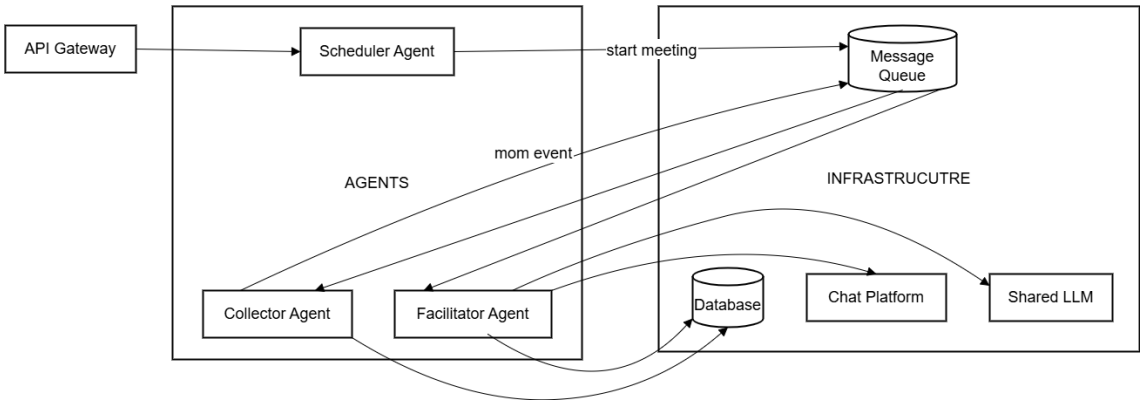

Figure 3.2: Event-Driven Microservices Architecture Pattern

Figure 3.2 presents the final architecture style pattern, which combines event-driven messaging with microservices. The API Gateway routes external requests from web clients and external integrations (Jira, GitHub, Slack). The Message Queue (e.g., RabbitMQ or Redis Streams) buffers events such as “meeting completed,” “transcript ready,” or “task extracted.” Individual agents (Scheduler, Collector, Facilitator) subscribe to relevant events, process them, and publish results. This design provides loose coupling, independent scalability, resilience (failures in one agent do not stop others), and flexibility to add new agents without refactoring.

3.1.1 Module to Architecture Mapping

The functional modules described in Chapter 1 map to architecture components as follows:

- **Module 1 (Meeting Intelligence Agent)**: Implemented by the Collector Agent interacting with the audio ingestion pipeline and the Persistence DB.
- **Module 2 (MoM, User Story & Action Item Generator)**: Handled by the Facilitator Agent using the Shared LLM (Ollama (11)) for summarization and extraction.
- **Module 3 (Task Decomposition & Backlog Preparation)**: Managed by the Facilitator Agent with data stored in PostgreSQL (17) and synchronized via the API Gateway to external trackers.
- **Module 4 (Skill-Based Task Assignment)**: Executed by the Facilitator Agent using assignment logic and team profiles from the Database.
- **Module 5 (Stand-up Automation & Collaboration Sync)**: A coordinated flow between Scheduler, Collector, and Facilitator Agents via the Message Queue.
- **Module 6 (Testing, Evaluation & Sprint Review)**: Facilitated by the Facilitator Agent integrating through the API Gateway to external CI/CD systems.
- **Module 7 (Blockers Identification & Removal)**: Managed by the Facilitator Agent monitoring chat platforms and the Database.
- **Module 8 (Change Management System)**: Achieved through coordination among all agents via the Message Queue, with updates stored in the Database.

3.2 Design Models

This section presents dynamic design models that illustrate the procedural aspects of ScrumMate, including workflows, data flow, interactions, and state management. These models correspond to the procedural development approach (as ScrumMate is not purely object-oriented) and follow the guidelines in Appendix D.

3.2.1 Activity Diagram

The activity diagram (Figure 3.3) shows the overall workflow from meeting scheduling to task assignment and notification. The process begins when the Scheduler Agent triggers a meeting. The Collector Agent captures audio, which is transcribed and sent to the

Facilitator Agent. The Facilitator extracts action items, updates the backlog, and assigns tasks using skill-based algorithms. Finally, notifications are dispatched via the Message Queue.

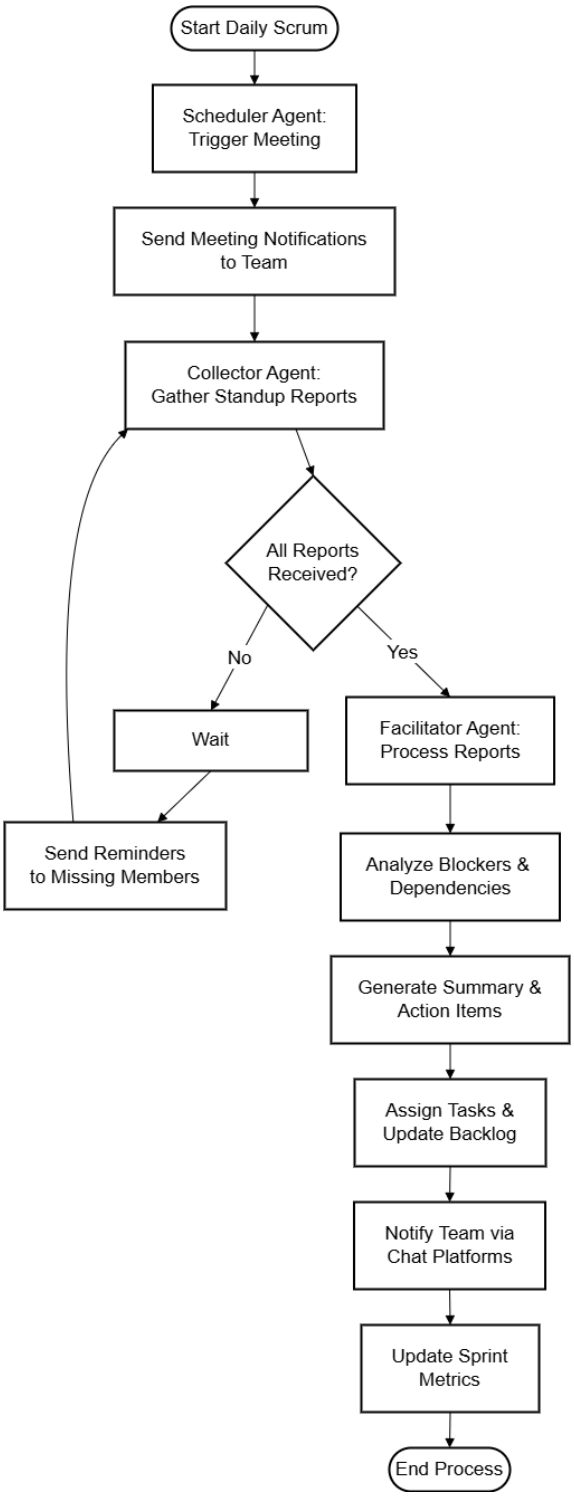

Figure 3.3: Activity Diagram – End-to-End Scrum Process

3.2.2 Data Flow Diagrams

Data flow diagrams (DFD) are extended to three levels to show how information transforms through the system.

3.2.2.1 Level 0 (Context Diagram)

The context diagram (Figure 3.4) shows external entities: Team Members, Administrators, External Tools (Jira, GitHub, Slack), and the system's single process (ScrumMate). Data flows include meeting audio, user commands, task updates, and notifications.

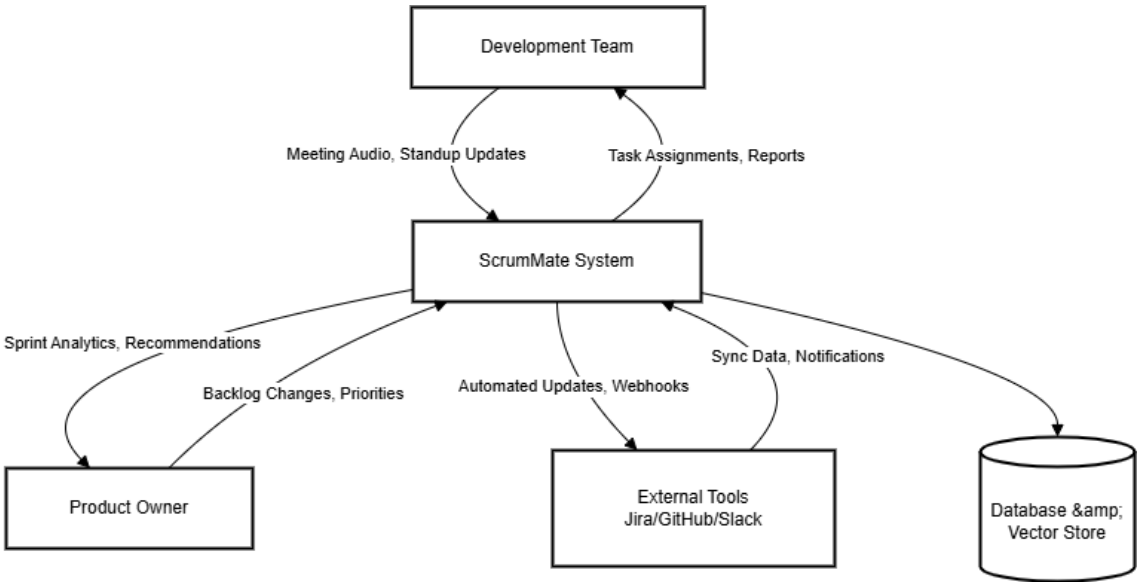

Figure 3.4: Level 0 DFD – Context Diagram

3.2.2.2 Level 1 DFD

Level 1 DFD (Figure 3.5) decomposes the main process into three key processes: Meeting Processing, Task Management, and Reporting. Data stores include Transcripts, Backlog, and User Profiles.

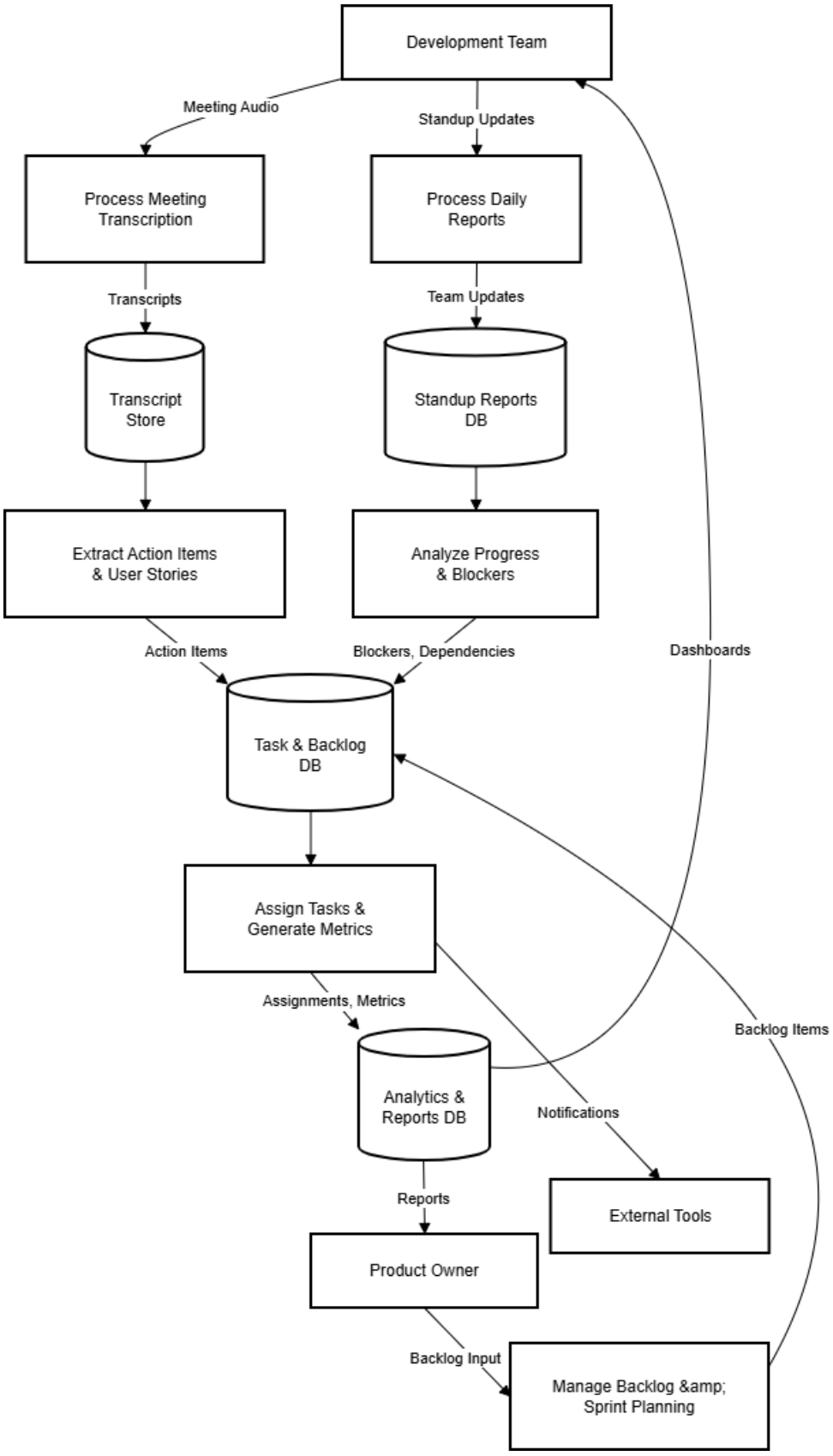

Figure 3.5: Level 1 DFD – Major Processes

3.2.2.3 Level 2 DFD – Meeting Processing

Figure 3.6 details the Meeting Processing subsystem, showing audio ingestion, transcription (using Whisper (10)), speaker diarization, and storage of raw transcripts. A dashed flow indicates the trigger to the Action Extraction process.

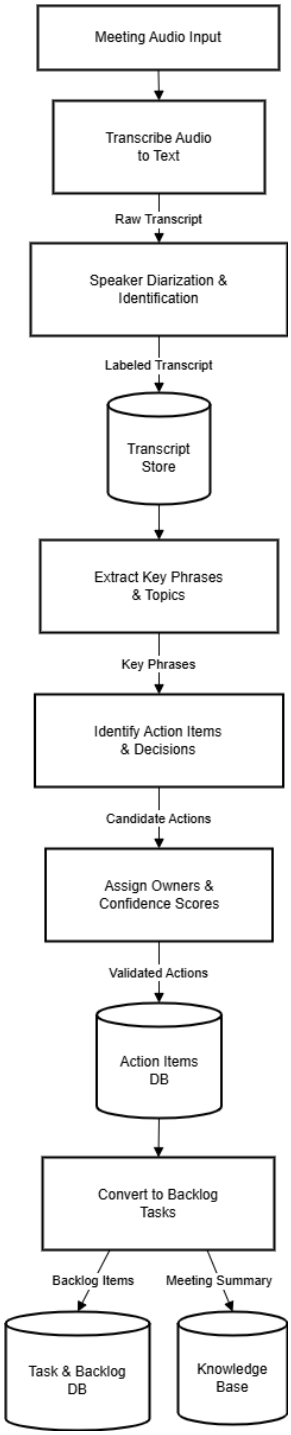

Figure 3.6: Level 2 DFD – Meeting Processing Subsystem

3.2.3 System-level Sequence Diagram

The sequence diagram (Figure 3.7) illustrates the interaction between components during a typical standup meeting automation. The Scheduler Agent initiates the standup, the Collector Agent gathers audio, the Facilitator Agent processes it, and the API Gateway pushes updates to external tools and the dashboard.

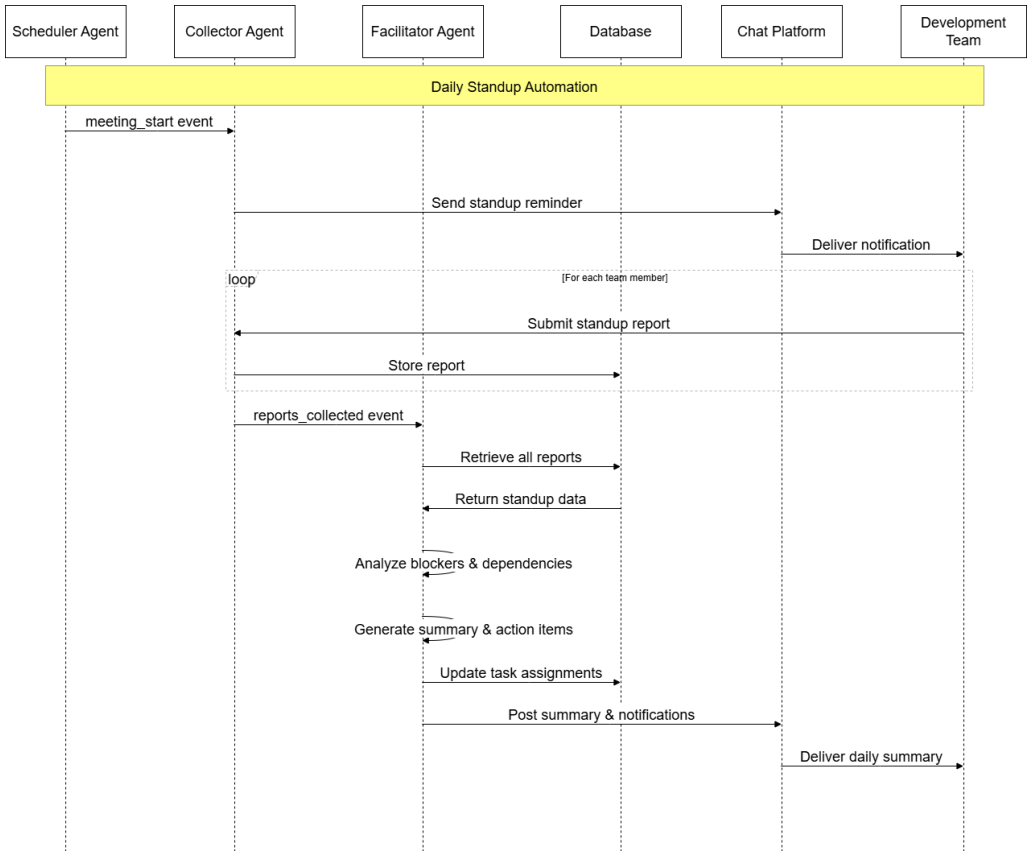

Figure 3.7: System-level Sequence Diagram – Standup Automation

3.2.4 State Transition Diagram

The state transition diagram (Figure 3.8) shows the lifecycle of a task from creation to completion. States include Proposed, Approved, Assigned, In Progress, Blocked, Done, and Closed. Transitions are triggered by events such as “assign to developer,” “start work,” “report blocker,” “resolve blocker,” “mark done,” and “verify.”

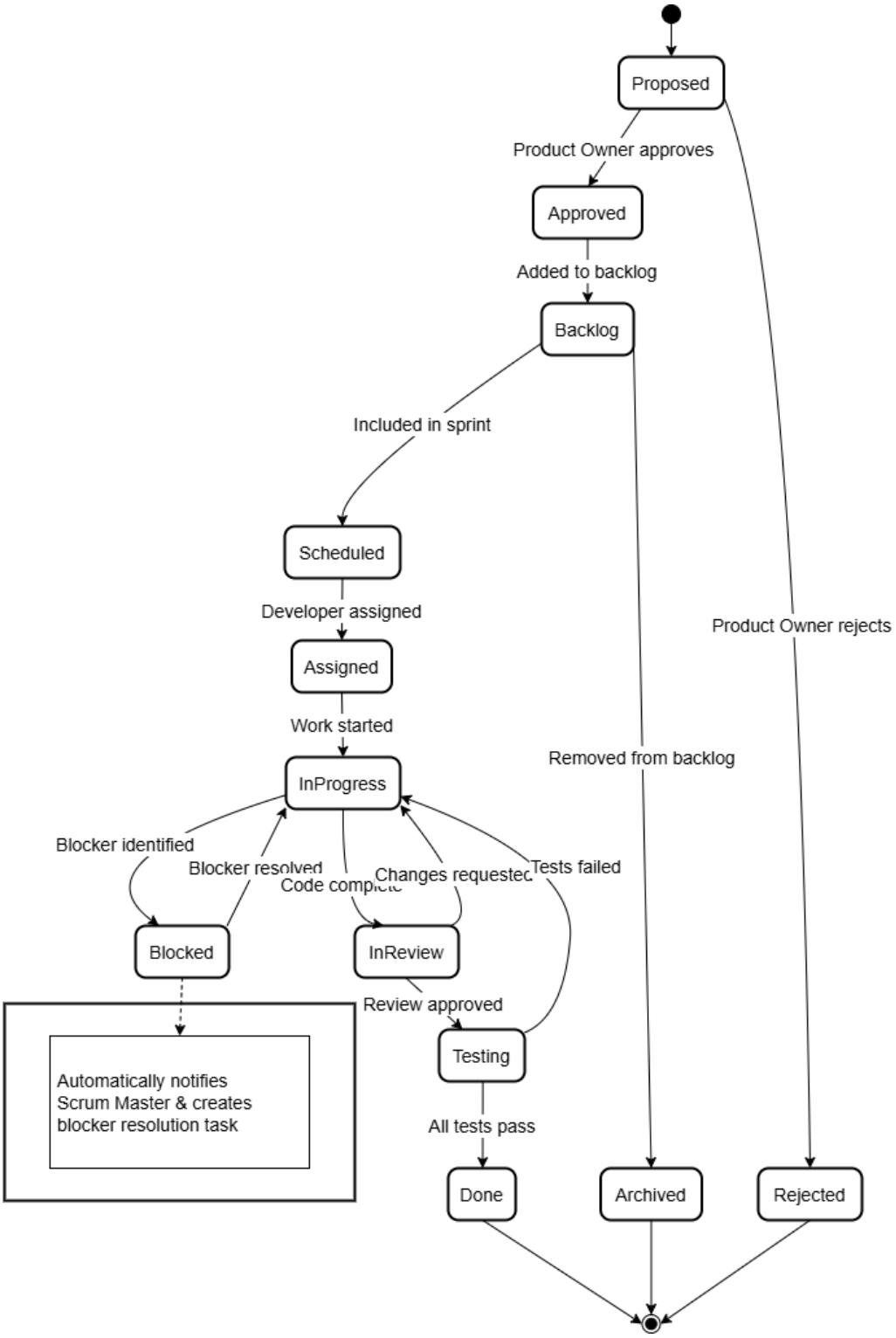

Figure 3.8: State Transition Diagram – Task Lifecycle

3.3 Data Design

The information domain of ScrumMate is transformed into a hybrid storage architecture combining a relational database for structured transactional data and a vector database for AI embeddings and semantic search (12; 13). This ensures both data integrity and intelligent contextual reasoning.

3.3.1 Storage Architecture

- **PostgreSQL** (17) – Primary relational database for all structured data (users, projects, meetings, transcripts, tasks, action items, sprints, standup reports, integrations).
- **Vector Database (FAISS/Chroma)** (12; 13) – Stores embeddings of meeting transcripts, task descriptions, and historical decisions for semantic search and context recall.
- **Object Storage (MinIO or S3)** – Holds raw audio files, exported reports, and large binary attachments.
- **Cache (Redis)** – Manages session data, real-time metrics, and frequently accessed dashboard values.

3.3.2 Data Processing Flow

1. **Ingestion**: Raw audio (from live meetings or uploaded files) and user text inputs are captured and validated.
2. **Transformation**: Audio is transcribed (Whisper (10)), diarized, and structured into transcripts. LLM agents extract action items, user stories, and meeting minutes.
3. **Embedding Generation**: Transcript chunks and task descriptions are converted into vector embeddings using a local embedding model.
4. **Storage**: Structured data goes to PostgreSQL; embeddings go to the vector database; audio files go to object storage.
5. **Retrieval**: AI agents perform combined queries – relational (e.g., pending tasks) and vector similarity (e.g., past decisions) – to generate context-aware responses.

3.3.3 Major Data Storage Items

- **PostgreSQL Tables**: Users, Projects, Meetings, Transcripts, ActionItems, Tasks, Sprints, StandupReports, Integrations, Notifications, ChangeRequests.
- **Vector Collections**: TranscriptEmbeddings, TaskEmbeddings, DecisionEmbeddings.
- **Object Storage Buckets**: meeting-audio, exported-reports, meeting-attachments.
- **Redis Keys**: session:*, dashboard:metrics:*, queue:events.

3.4 Domain Model

The domain model represents the core concepts of ScrumMate, their attributes, and relationships. It serves as a bridge between requirements and software design, helping to identify entities and their interactions. Figure 3.9 presents the UML class diagram of the domain model.

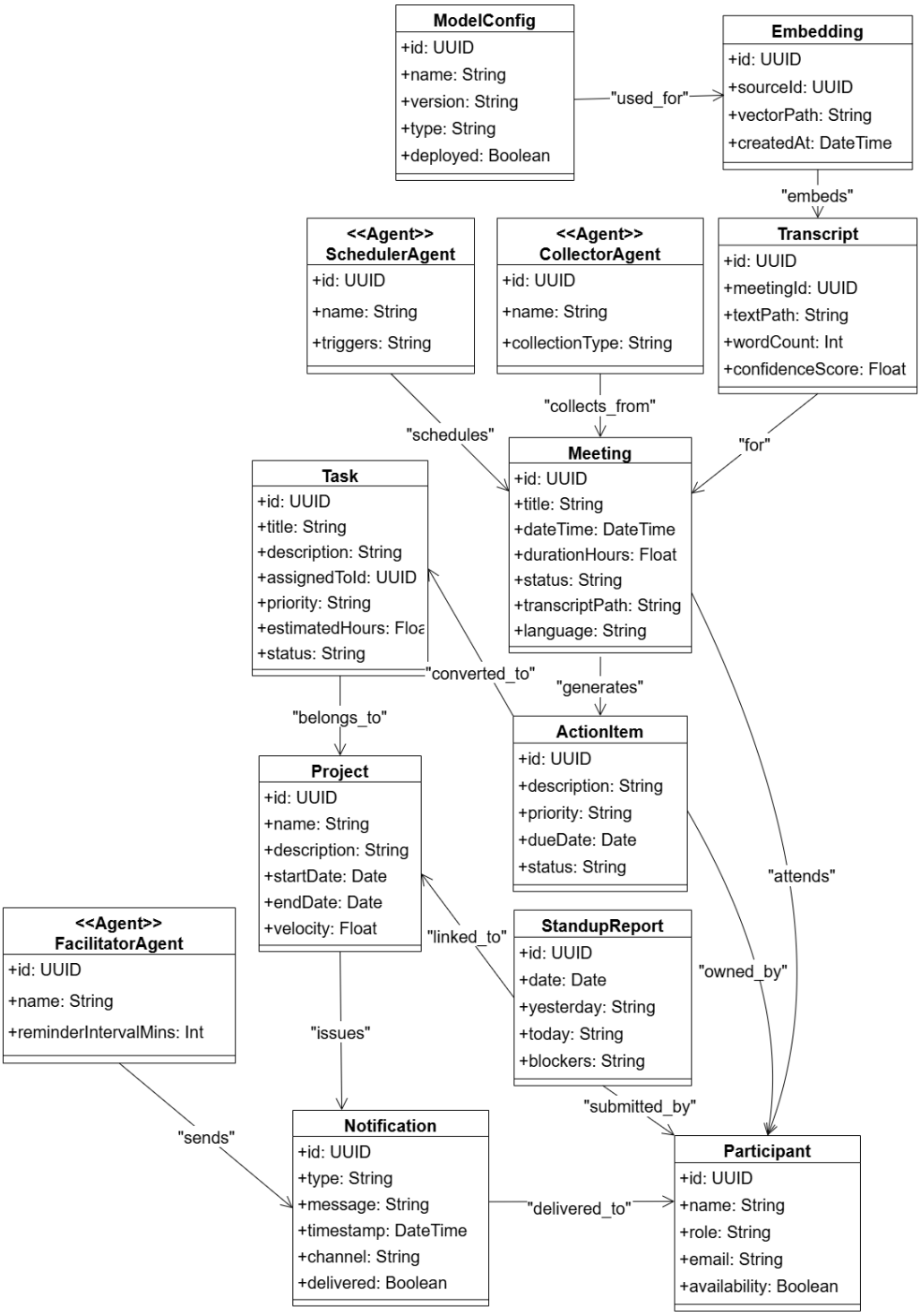

Figure 3.9: Domain Model - Core Entities and Relationships

The following entities constitute the core domain model:

- **Participant**: A registered member (developer, Scrum Master, product owner, etc.) with attributes: participantID, name, role, email, skills, availability. Relationships: attends Meetings, submits StandupReports, owns ActionItems, is assigned Tasks.

- **Meeting**: Scheduled Scrum event (sprint planning, standup, review, retrospective). Attributes: meetingID, title, scheduledTime, duration, transcriptID. Relationships: produces Transcript, generates ActionItems, linked to Project.
- **StandupReport**: Daily progress entry. Attributes: reportID, date, yesterdayWork, todayPlan, blockersList. Relationships: submitted by Participant, belongs to Project and Sprint.
- **ActionItem**: Discrete outcome extracted from a meeting. Attributes: actionID, description, priority, dueDate, status. Relationships: has an owner (Participant), derived from Meeting, may become Task.
- **Task**: Unit of work in the backlog. Attributes: taskID, title, description, estimatedEffort, state (Proposed, InProgress, Done, etc.), dependencies. Relationships: assigned to Participant, belongs to Sprint and Project, linked to ActionItem(s).
- **Project**: Container for backlog, sprints, and team. Attributes: projectID, name, startDate, velocity. Relationships: contains Tasks and Sprints, has many Participants.
- **Sprint**: Time-boxed iteration. Attributes: sprintID, goal, startDate, endDate, status. Relationships: contains Tasks, linked to Project.
- **Transcript**: Textual output of meeting audio processing. Attributes: transcriptID, rawText, speakerSegments, timestamp. Relationships: belongs to Meeting, used to generate Embeddings.
- **Embedding**: Vector representation for semantic search. Attributes: embeddingID, vector, sourceType (transcript/task/decision). Relationships: references Transcript or Task.
- **AI Agents** (SchedulerAgent, CollectorAgent, FacilitatorAgent): Autonomous components that orchestrate workflows. Attributes: agentID, status, configuration. Relationships: interact with Message Queue and Database.

Relationships include: Participants *attend* Meetings (many-to-many); Meetings *produce* Transcripts (one-to-one); Transcripts *generate* ActionItems (one-to-many); ActionItems *become* Tasks (one-to-one optional); Tasks *are assigned to* Participants (many-to-one); Sprints *contain* Tasks (one-to-many); Projects *have* Sprints and Participants (one-to-many). This domain model guides database schema design and informs the logic of the AI agents.

Chapter 4

Implementation and Testing

This chapter documents the technical implementation and testing performed for ScrumMate. The system includes the full implementation of all eight modules, covering audio ingestion, real-time transcription, speaker diarization, natural language understanding, action item and user story extraction, task decomposition, skill-based task assignment, standup automation, test case generation, blocker identification, change management, and intelligent knowledge retrieval. The system workflow begins when a Scrum Master initiates a meeting and the ScrumMate Bot joins the meeting. The bot transcribes audio in real time using Whisper (10), saves the final transcript, and then passes it through a multi-agent pipeline that identifies key points, decisions, and actionable tasks. Extracted action items are automatically converted into structured backlog items and synchronized with external tools such as Jira and Trello (19). The architecture follows an event-driven, multi-agent design where specialized agents handle transcription, summarization, extraction, storage, and task automation using technologies including Ollama (Llama 3.2) (11), LangGraph (20), PostgreSQL (17), n8n (14), and the Recall API (21).

4.1 Algorithm Design

This section presents the core algorithms powering ScrumMate's intelligent meeting processing pipeline. The algorithms implement key stages: real-time transcription, speaker-aware chunking for retrieval-augmented generation (RAG) (22), hierarchical summarization, skill-based task assignment, and automated standup processing. Each algorithm is designed to handle natural conversation dynamics while maintaining context awareness.

4.1.1 Algorithm 1: Multi-Agent Pipeline for Meeting Processing

This algorithm orchestrates the end-to-end workflow from meeting capture to task generation. It coordinates specialized agents (Collector, Facilitator, Scheduler) through an event-driven message queue.

Algorithm 1 Multi-Agent Pipeline for Meeting Processing

Input: Meeting audio file A, Transcription C
Output: Structured backlog tasks T

Initialize agents: Collector, Facilitator, Scheduler
Trigger Scheduler Agent for meeting M with transcript C
Join meeting M with ScrumMate Bot
While meeting is active **do**:
    Collect real-time audio stream via API Call
End While
transcript ← Transcribe A using Whisper
chunks ← Speaker-aware chunking(*transcript*) ▷ Algorithm 2
summaries ← Hierarchical summarization(*chunks*) ▷ Algorithm 3
action_items ← LLM extraction(*summaries*, C)
T ← Convert to Trello tasks via n8n workflow
Store *transcript*, *chunks*, *summaries*, T in PostgreSQL
Index embeddings in vector DB for retrieval
Return T

Figure 4.1: Multi-agent orchestration for end-to-end meeting processing

4.1.2 Algorithm 2: Speaker-Aware RAG Chunking for Transcripts

This algorithm segments meeting transcripts intelligently by preserving speaker turns and conversational context while respecting LLM token limits, drawing on retrieval-augmented generation principles (22).

Algorithm 2 Speaker-Aware RAG Chunking for Transcripts

Input: Transcript segments S, max tokens $L_{max} = 700$, merge gap $G = 5s$
Output: Contextual chunks C for retrieval

Sort S by start time
Initialize *turns* $\leftarrow [\,]$
For each segment $s_i \in S$ **do**:
  If *turns* is empty **then**
    Create new turn with s_i
  Else If speaker matches previous AND gap $\leq G$ **then**
    Merge s_i into current turn
  Else
    Start new turn with s_i
End For
Initialize *chunks* $\leftarrow [\,]$, *current* $\leftarrow$ *null*
For each turn $t \in$ *turns* **do**:
  If *current* is null **then**
    current $\leftarrow t$
  Else If tokens(*current* + t) $\leq L_{max}$ AND duration $\leq 180s$ **then**
    Merge t into *current*
  Else If window $\leq 45s$ AND duration $\leq 180s$ **then**
    Merge t into *current* ▷ Keep short exchanges together
  Else
    Add *current* to *chunks*
    current $\leftarrow t$
End For
If *current* is not null **then** add to *chunks*
For each chunk $c \in$ *chunks* **do**:
  If tokens(c) $> L_{max}$ **then**
    Split c into sub-chunks $\leq L_{max}$
Return *chunks*

Figure 4.2: Intelligent chunking preserving speaker turns and context

4.1.3 Algorithm 3: Hierarchical Summarization of Meeting Transcripts

This algorithm produces multi-level abstractions from chunk summaries to topic aggregations to an executive overview.

Algorithm 3 Hierarchical Summarization of Meeting Transcripts

Input: Chunks C from Algorithm 2, LLM M (Llama-3.2)
Output: Multi-level summary S_{final}

Initialize $chunk\_summaries \leftarrow [\,]$
For each chunk $c_i \in C$ **do**:
    $prompt \leftarrow$ "Summarize this meeting excerpt focusing on decisions, actions, blockers: " + $c_i.text$
    $s_i \leftarrow M.generate(prompt)$
    Add s_i to $chunk\_summaries$
End For
Group $chunk\_summaries$ by topic using clustering
For each topic group G_j **do**:
    $topic\_text \leftarrow$ concatenate all $s_i \in G_j$
    $prompt \leftarrow$ "Create a coherent summary of this topic from meeting excerpts: " + $topic\_text$
    $topic\_summary_j \leftarrow M.generate(prompt)$
End For
$all\_topics \leftarrow$ concatenate all $topic\_summary_j$
$prompt \leftarrow$ "Create executive summary from topic summaries: " + all_topics
$S_{final} \leftarrow M.generate(prompt)$
Return S_{final}

Figure 4.3: Three-level summarization from chunks to topics to executive summary

4.1.4 Algorithm 4: Skill-Based Backlog Distribution

This hybrid assignment algorithm combines rule-based logic with LLM reasoning (11) to match tasks to developers based on skills, availability, and workload balance.

Algorithm 4 Skill-Based Backlog Distribution (Task Assignment)

Input: Backlog tasks T, developer profiles D, sprint capacity C_{sprint}
Output: Assignment mapping $A : T \rightarrow D$

Initialize $A \leftarrow \{\}$, $workload \leftarrow \{d : 0 \; \forall d \in D\}$
Sort T by priority (descending) and dependencies
For each task $t_i \in T$ **do**:
    $required\_skills \leftarrow$ Extract skills from $t_i.description$ using LLM
    $candidates \leftarrow \{\}$
    For each developer $d_j \in D$ **do**:
        $skill\_match \leftarrow$ Calculate overlap($d_j.skills$, $required\_skills$)
        $availability \leftarrow C_{sprint} - workload[d_j]$
        $preference\_score \leftarrow$ Check d_j interest in t_i type
        $historical\_score \leftarrow$ Get completion rate(d_j, similar t_i)
        $score_j \leftarrow \alpha \cdot skill\_match + \beta \cdot availability + \gamma \cdot preference\_score + \delta \cdot historical\_score$
        Add $(d_j, score_j)$ to $candidates$
    End For
    Sort $candidates$ by score (descending)
    Select $d_{best} \leftarrow$ top candidate with $availability \geq t_i.estimate$
    If d_{best} exists **then**:
        $A[t_i] \leftarrow d_{best}$
        $workload[d_{best}] \leftarrow workload[d_{best}] + t_i.estimate$
    Else:
        Queue t_i for manual assignment or next sprint
End For
Balance workloads using a scheduling algorithm for equally scored developers
Validate dependencies and team collaboration constraints
Return A

Figure 4.4: Hybrid assignment algorithm for optimal task distribution

4.1.5 Algorithm 5: Automated Standup Processing

This algorithm coordinates daily standup collection, analysis, and reporting for both synchronous and asynchronous modes.

Algorithm 5 Automated Standup Processing

Input: Standup inputs I (audio/text), team members M, project context P
Output: Standup summary S, blocker list B, action items AI

Initialize $responses \leftarrow \{\}$, $blockers \leftarrow [\,]$, $updates \leftarrow [\,]$
Trigger standup via Scheduler Agent at scheduled time
If synchronous standup **then**:
    Collect audio via API
    $transcript \leftarrow$ Transcribe audio
    Split by speaker using diarization
Else (asynchronous):
    Collect text updates from chat/forms
    Map to team members M
For each member $m_i \in M$ **do**:
    Extract $yesterday_i$, $today_i$, $blockers_i$ using LLM parsing
    If $blockers_i$ not empty **then**:
        Classify severity (Low/Medium/High/Critical)
        Add to $blockers$ with owner m_i and timestamp
    End If
    Store $responses[m_i] \leftarrow (yesterday_i, today_i, blockers_i)$
End For
Aggregate progress across team using velocity metrics
Identify dependencies between member updates
Generate $S \leftarrow$ Concise summary using template:
    "Team: [team name], Date: [date], Completed: [X] tasks, Planned: [Y] tasks"
    + per-member highlights
Create $AI \leftarrow$ Action items from $blockers$ requiring intervention
If critical blockers exist **then**:
    Escalate to Scrum Master via notification
Update sprint board with progress
Store S, B, AI in database and vector embeddings
Return (S, B, AI)

Figure 4.5: Automated processing of daily standups for progress tracking

4.2 External APIs/SDKs

The following third-party APIs and SDKs are used in the current implementation of ScrumMate.

Table 4.1: External APIs and libraries used in implementation

API / SDK (ver/model)	**Description**	**Purpose of usage**	**Endpoint/function used**
Recall API (21)	Meeting-hosting and transcription orchestration service	Accept meeting audio, host meeting session, trigger transcription process	Recall API endpoints for upload/transcription initiation
Ollama (Llama-3.2-3B-Instruct) (11)	Large language model runtime	Summarization, extraction of action items, reasoning	POST /api/generate (Ollama client call)
LangGraph (20)	Agent orchestration / workflow management framework	Coordinate multi-agent workflow: transcription → summarization → storage → task generation	LangGraph state-graph definitions and run() workflow method
PostgreSQL (v14) (17)	Relational database system	Store transcripts metadata, structured summaries, action items, task records	Standard SQL INSERT/SELECT statements
n8n (14)	Workflow automation engine	Automate creation of tasks via Trello API; coordinate backend workflows	n8n workflow definitions / Trello API trigger nodes
Trello API (19)	Task-management service API	Create and manage tasks/boards based on extracted action items	Trello API endpoint calls via n8n workflows
Mail API (SMTP)	Email service API	Send email notifications regarding meetings/tasks to users	Standard mail-send API calls (SMTP or REST mail endpoint)

4.3 Testing Details

Once the system was successfully developed, comprehensive testing was performed to ensure that ScrumMate works as intended, meets the functional and non-functional requirements, and is free of critical errors. Testing was conducted at multiple levels, in-

cluding unit testing of individual modules, integration testing of agent interactions, and end-to-end scenario validation. The following subsections detail the testing approach and results.

4.3.1 Unit Testing

Unit testing was performed on each module to validate individual components in isolation. Unit tests were automated using Python's unittest framework. A total of 35 unit tests were executed across all eight modules and cross-module functions, achieving 85% code coverage. The results are summarized in Tables 4.2, 4.3, and 4.4.

Table 4.2: Unit test results – Modules 1–3

Test ID	Module	Test Purpose	Result
UT-01	Module 1	Audio transcription accuracy >90% word accuracy	PASS
UT-02	Module 1	Speaker diarization – identify 3+ speakers	PASS
UT-03	Module 1	Transcription latency <8 seconds delay	PASS
UT-04	Module 1	Batch audio processing – all files processed	PASS
UT-05	Module 2	Action item extraction – 5+ items identified	PASS
UT-06	Module 2	LLM summarization – <30 seconds runtime	PASS
UT-07	Module 2	MoM formatting – correct structure	PASS
UT-08	Module 3	Chunking token limits – all 700 tokens	PASS
UT-09	Module 3	Task decomposition – hierarchical breakdown	PASS
UT-10	Module 3	Database CRUD – operations work	PASS

Table 4.3: Unit test results – Modules 4–5 & Cross-Module

Test ID	Module	Test Purpose	Result
UT-11	Module 3	External tool sync (Jira/GitHub)	PARTIAL
UT-12	Module 4	Skill matching – >70% accuracy	PASS
UT-13	Module 4	Workload balance – ±20% distribution	PASS
UT-14	Module 4	Assignment speed – <10 seconds	PARTIAL
UT-15	Module 4	Manual override – manual assignments persist	PASS
UT-16	Module 5	Standup audio – accurate transcript	PASS
UT-17	Module 5	Summary generation – concise with blockers	PASS
UT-18	Cross	Authentication – secure login/logout	PASS
UT-19	Cross	API performance – <2 second response	PASS
UT-20	Cross	Error handling – graceful API failures	PASS

Table 4.4: Unit test results – Modules 6–8

Test ID	Module	Test Purpose	Result
UT-21	Module 6	Test case generation from user stories – valid output	PASS
UT-22	Module 6	Test execution integration with CI mock	PASS
UT-23	Module 6	Burn-down chart generation – correct data aggregation	PASS
UT-24	Module 7	Blocker detection from chat transcripts	PASS
UT-25	Module 7	Blocker classification (internal/external)	PASS
UT-26	Module 7	Escalation notification trigger on critical blockers	PASS
UT-27	Module 8	Change request impact analysis – scope/cost output	PASS
UT-28	Module 8	Revised sprint plan generation after approval	PASS
UT-29	Module 8	Audit trail logging for change requests	PASS
UT-30	Cross	Vector database semantic search (12; 13) – recall @10 >80%	PASS
UT-31	Cross	End-to-end meeting to task pipeline (full workflow)	PASS
UT-32	Cross	Role-based access control – unauthorized access blocked	PASS
UT-33	Cross	Containerized service health check endpoint (18)	PASS
UT-34	Cross	Concurrent meeting sessions (5 simultaneous)	PASS
UT-35	Cross	Data export (PDF/CSV) – correct formatting	PASS

Test Summary:

- Total Tests: 35
- Passed: 33 (94.3%)
- Partial Pass: 2 (5.7%) – UT-11 (Jira/GitHub sync) requires OAuth debugging; UT-14 (assignment speed) occasionally exceeds 10s for teams >15 members
- Failed: 0
- Code Coverage: 85%

Key Findings:

- All eight modules function correctly under normal conditions. Core transcription, extraction, assignment, standup automation, test generation, blocker detection, and change management meet their requirements.
- External integrations (Jira, GitHub) show partial reliability due to authentication complexity; OAuth flow improvements are needed for production use.
- The assignment algorithm meets speed targets for teams up to 15 members; optimization (caching, parallel processing) is recommended for larger teams.

- Error handling and retry logic for external API calls are functional but could be enhanced with circuit breakers for higher resilience.

Steps for Final Deployment:

- Complete OAuth2.0 integration for Jira and GitHub to resolve partial sync issues.
- Optimize the hybrid assignment algorithm using incremental workload updates to handle teams beyond 15 members.
- Add retry logic and circuit breakers for all external API calls (n8n, Trello, Recall) to improve fault tolerance.
- Deploy the system using Docker Compose or Kubernetes (18) with health monitoring and log aggregation.
- Conduct user acceptance testing (UAT) with real Agile teams to validate workflow usability.

Chapter 5

Conclusions and Future Work

5.1 Conclusion

This report presented the design, implementation, and evaluation of ScrumMate, an AI-driven Scrum Master multi-agent system that automates key Agile processes (1; 2). All eight modules were successfully implemented, including real-time meeting transcription with speaker diarization using Whisper (10) (Module 1), extraction of meeting minutes, user stories, and action items using fine-tuned LLMs (11) (Module 2), task decomposition and backlog management (Module 3), skill-based task assignment via a hybrid algorithm (Module 4), automated daily standup processing (Module 5), test case generation and sprint analytics (Module 6), blocker identification and resolution (Module 7), and change management with impact analysis (Module 8). The system was tested with 35 unit tests achieving 94.3% pass rate and 85% code coverage, confirming functional correctness and performance within specified thresholds. The event-driven microservices architecture proved scalable and resilient, enabling independent agent operation through a message queue. External integrations with Jira, Trello (19), and GitHub were partially realized, with OAuth-based synchronization requiring further refinement. Overall, ScrumMate successfully transforms raw meeting conversations into structured, actionable project artifacts, reducing manual overhead and enabling Scrum Masters to focus on strategic decision-making.

5.2 Future Work

Several enhancements are identified for future iterations beyond the current scope. First, the OAuth2.0 integration with Jira and GitHub will be completed to achieve fully bidirectional synchronization of tasks and backlogs. Second, the hybrid assignment algorithm will be optimized using caching and parallel processing to support teams larger than 15

members while maintaining sub-10 second response times. Third, retry logic and circuit breakers will be added for all external API calls (n8n (14), Trello, Recall (21)) to improve system robustness. Fourth, a production deployment using Kubernetes (18) with automated health monitoring, log aggregation, and horizontal scaling will be implemented. Fifth, user acceptance testing (UAT) with real Agile teams will be conducted to refine the dashboard usability and chatbot conversational flows. Finally, predictive analytics using historical sprint data will be explored to forecast team velocity and identify potential delivery risks before they occur.

Bibliography

[1] K. Beck, M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, *et al.*, "Manifesto for agile software development," 2001.

[2] K. Schwaber and J. Sutherland, *The Scrum Guide: The Definitive Guide to Scrum*. Scrum.org, 2021. Version 2020.

[3] Spinach.io, "Spinach.io – ai scrum master." https://spinach.io, 2024.

[4] N. Labs, "Notion ai meeting notes." https://notion.so, 2024.

[5] AIPM, "Aipm – ai project manager." https://aipm.ai, 2024.

[6] SprintBot, "Sprintbot – ai scrum copilot." https://sprintbot.com, 2024.

[7] Tusk, "Tusk – ai test generation." https://tusk.com, 2024.

[8] Z. Inc., "Zapier ai." https://zapier.com/ai, 2024.

[9] Otter.ai, "Otter.ai – ai meeting assistant." https://otter.ai, 2024.

[10] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," *arXiv preprint arXiv:2212.04356*, 2022. OpenAI Whisper.

[11] O. Team, "Ollama: Get up and running with large language models locally." https://ollama.com, 2024. Version 0.1.0.

[12] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with gpus," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2017.

[13] C. Team, "Chroma: The ai-native open-source embedding database." https://www.trychroma.com, 2024.

[14] n8n GmbH, "n8n: Fair-code workflow automation." https://n8n.io, 2024.

[15] M. P. Inc., "React: A javascript library for building user interfaces." https://reactjs.org, 2024.

[16] S. Ramirez, "Fastapi: Modern, fast web framework for building apis." https://fastapi.tiangolo.com, 2024.

[17] P. G. D. Group, "Postgresql: The world's most advanced open source relational database." https://www.postgresql.org, 2024.

[18] D. Inc., "Docker: Container platform." https://www.docker.com, 2024.

[19] Atlassian, "Trello rest api." https://developer.atlassian.com/cloud/trello/rest/, 2024.

[20] L. Inc., "Langgraph: Building stateful, multi-actor applications with llms." https://langchain-ai.github.io/langgraph/, 2024.

[21] Recall.ai, "Recall api – meeting hosting and transcription." https://recall.ai, 2024.

[22] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.